AMRITA VISHWA VIDYAPEETHAM
Department of Computer Science and Engineering

Improving the Weisfeiler–Lehman Kernel with Log-Binned Degree Labels and an RBF Kernel

Weisfeiler–Lehman Graph Kernels on REDDIT-BINARY

Term Project Report

Course	Machine Learning with Graphs (23CSE479)
Team Members	Sidharth S Nair — CB.SC.U4CSE23443
	Yash Bardia — CB.SC.U4CSE23458
	S Sanjeev Srinivas — CB.SC.U4CSE23466
	Karthik P — CB.SC.U4CSE23522
Faculty Supervisor	Ramraj T
Dataset	REDDIT-BINARY (2,000 graphs; TUDataset format)

October 15, 2026

Contents

Abstract **1**

1 Introduction **1**

2 Problem Statement **1**

3 Objectives **1**

4 Dataset and Classification Labels **2**
4.1 REDDIT-BINARY 2
4.2 Classification labels 2
4.3 Dataset statistics 2

5 The Weisfeiler–Lehman Kernel **3**
5.1 Algorithm 3
5.2 What basic WL does to REDDIT-BINARY 4
5.3 Limitations of the raw-degree baseline on Reddit 5

6 Improvement 1: Log-Binned Degree Labels **6**
6.1 Idea 6
6.2 Why it helps 6

7 Improvement 2: Comparing WL Histograms - RBF Kernel **7**
7.1 RBF kernel on WL features 7
7.2 Comparison of the kernels 8
7.3 Why the two changes fit together 8

8 Implementation and Evaluation Protocol **9**
8.1 Implementation and correctness testing 9
8.2 Nested cross-validation 9
8.3 Significance test 10

9 Final Results: Baseline to Best, One Change at a Time **10**
9.1 What the results show 10
9.2 Confusion matrices 11
9.3 What the kernels see 12

10 Conclusion **12**

Abstract

This project improves the Weisfeiler–Lehman (WL) subtree kernel for classifying Reddit threads as either question-and-answer style or discussion style. Each thread is a graph in which nodes are users and edges mean "replied to". The graphs carry no node labels, so the baseline kernel initialises each node with its degree and compares the resulting label-count vectors with a linear kernel. In our nested cross-validation this baseline reaches 81.75% accuracy.

We make two small, explainable changes that keep the WL algorithm itself untouched. First, we group degrees into logarithmic ranges (1, 2–3, 4–7, 8–15, . . .) so that similar nodes in different graphs receive matching labels. Second, we change the way the WL label counts are compared: we study cosine normalisation and an RBF kernel, and adopt the RBF kernel. Together the two changes raise accuracy to 90.72%(+8.97 points).

1. Introduction

Graph kernels let a standard classifier such as a support vector machine (SVM) work on whole graphs by defining a similarity score between two graphs. The Weisfeiler–Lehman (WL) subtree kernel repeatedly updates each node's label using its current label and the labels of its neighbours. It then compares graphs by counting the structural patterns they share across successive iterations.

REDDIT-BINARY is a difficult setting for WL because its graphs have no node labels. Everything WL can learn comes from (a) the starting labels chosen (b) how the final label counts are compared. This report studies exactly those two places. We first describe the dataset and the classification task, then show what the basic WL kernel does to it, and finally present and evaluate our two improvements: **log-binned degree labels** and an **RBF kernel** on the WL histograms.

2. Problem Statement

Given a Reddit thread represented only by its reply graph, predict whether it comes from a *question-and-answer* (Q&A) community (IAmA, AskReddit) or a *discussion* community (TrollXChromosomes, atheism). The standard WL recipe for unlabelled graphs gives starting labels that are too specific to repeat across graphs, and a dot-product comparison lets large graphs dominate. The challenge is to fix both without leaving the WL framework and without losing explainability.

3. Objectives

- Re-implement the WL subtree kernel from scratch and verify it against the textbook examples.
- Improve the starting labels so that WL labels are shared between graphs.
- Improve the way two WL label-count vectors are compared.

- Measure each change separately with nested cross-validation and a significance test.

4. Dataset and Classification Labels

4.1 REDDIT-BINARY

REDDIT-BINARY (Yanardag and Vishwanathan, 2015, distributed with TUDataset) contains **2,000 Reddit threads** drawn from four popular subreddits: two question-and-answer communities (IAmA, AskReddit) and two discussion communities (TrollXChromosomes, atheism). Each thread is one graph:

- a **node** is a user who took part in the thread;
- an **edge** $\{u, v\}$ means one of the two users replied to the other.

In total there are 859,254 nodes and 995,508 undirected edges. There are no node labels and no text: the task is to predict the community type from the shape of the conversation alone.

4.2 Classification labels

The task is binary classification with 1,000 graphs per class, so the classes are perfectly balanced. The dataset files do not document which numeric label corresponds to which community type, so we report the classes simply as -1 and $+1$.

4.3 Dataset statistics

Statistic	**Class** -1	**Class** $+1$
Graphs	1,000	1,000
Average nodes	641.3	218.0
Average edges	736.0	259.6
Average degree	2.41	2.27
Average maximum degree	336.3	98.4
Hub share (max degree / nodes)	0.508	0.460
Edge density	0.012	0.032
Trees ($\lvert E\rvert = \lvert V\rvert - 1$)	0.1%	5.9%

Table 1. Dataset statistics by class (computed in the notebook).

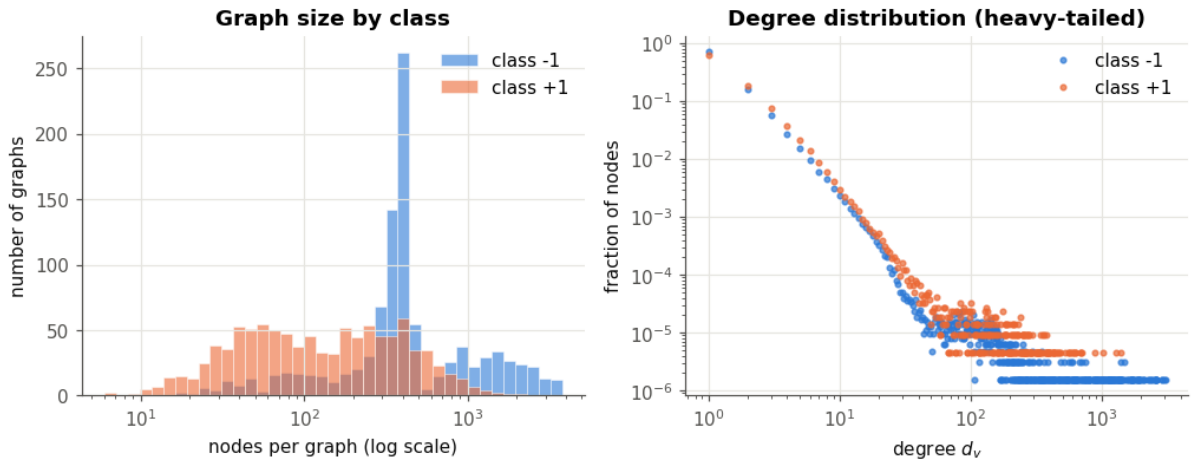


Figure 1. Graph size by class (left) and heavy-tailed degree distribution (right).

Three observations from the raw data drive the design.

- Class -1 threads are about three times larger than class $+1$ threads, so **size alone carries signal**.
- Degrees are **heavy-tailed**: most users reply once, while a few hub users, such as the original poster, connect to hundreds of others. This is the key to the initial-label improvement.
- The graphs are **tree-like with very few triangles**: edges $\approx$ nodes, and only 7.99% of nodes lie on a triangle.

5. The Weisfeiler–Lehman Kernel

5.1 Algorithm

WL colour refinement (Hamilton, §2.1.2 and §7.3.3) has four steps.

1. **Initial labels.** Give every node a starting label $l^{(0)}(v)$. For unlabelled graphs, the textbook baseline uses the node degree: "in most graphs, this label is simply the degree".
2. **Refinement.** For $i = 1, \ldots, h$, give each node a new label by hashing its own label together with the multiset of its neighbours' labels. The hash is a dictionary shared by all graphs, so the same neighbourhood pattern receives the same label in every graph.
3. **Feature map.** Count how often each label occurs in each graph to form a histogram $\phi(G)$.
4. **Comparison.** Compare two graphs by how many labels they share.

$$l^{(i)}(v) = \text{HASH}\left(l^{(i-1)}(v),\ \{\!\{ l^{(i-1)}(u) : u \in \mathcal{N}(v) \}\!\}\right) \qquad \text{(Hamilton Eq. 7.60)} \qquad (1)$$

The hash must be *injective*: two nodes receive the same new label if and only if they had the same old label and the same multiset of neighbour labels. After i iterations, $l^{(i)}(v)$ encodes the

rooted subtree of depth i around v. The feature map and the kernel are

$$\phi(G) = \Big[c(G,\sigma)\Big]_{\sigma\in\Sigma_0\cup\cdots\cup\Sigma_h}, \qquad c(G,\sigma) = \big|\{v \in V_G : l^{(i)}(v) = \sigma\}\big|, \tag{2}$$

$$k^{(h)}_{WL}(G,G') = \langle\phi(G),\phi(G')\rangle = \sum_{i=0}^{h}\sum_{\sigma\in\Sigma_i} c(G,\sigma)\,c(G',\sigma). \tag{3}$$

Stacking the histograms of all graphs as the rows of a matrix Φ gives the whole Gram matrix as $K = \Phi\Phi^\top$, which is passed to a kernel SVM with decision function $f(G) = \sum_j \alpha_j y_j k(G_j,G) + b$.

5.2 What basic WL does to REDDIT-BINARY

On REDDIT-BINARY the baseline kernel uses degree as the starting label, Eq. 7.60 as the update rule and the linear kernel. Every thread becomes a histogram counting how many of its nodes have each rooted-subtree pattern, and two threads are similar when they contain many of the same patterns. Table 2 shows the accuracy of this baseline for each depth h (10-fold nested cross-validation, one repetition).

WL depth h	**Accuracy (%)**	$\pm$ **std**	**F1**
0 (bag of nodes)	76.45	2.39	0.77
1	**81.05**	2.67	0.82
2	78.20	2.51	0.79
3	77.65	2.50	0.79
4	77.50	2.66	0.79

Table 2. Baseline WL (degree labels, linear kernel) by depth.

The baseline kernel reaches about 81%, best at $h = 1$. Going deeper makes it *worse*, from 81.1% to 77.5% at $h = 4$. With raw-degree labels each extra iteration creates labels so specific that almost no other graph shares them, so the added features contribute noise rather than similarity.

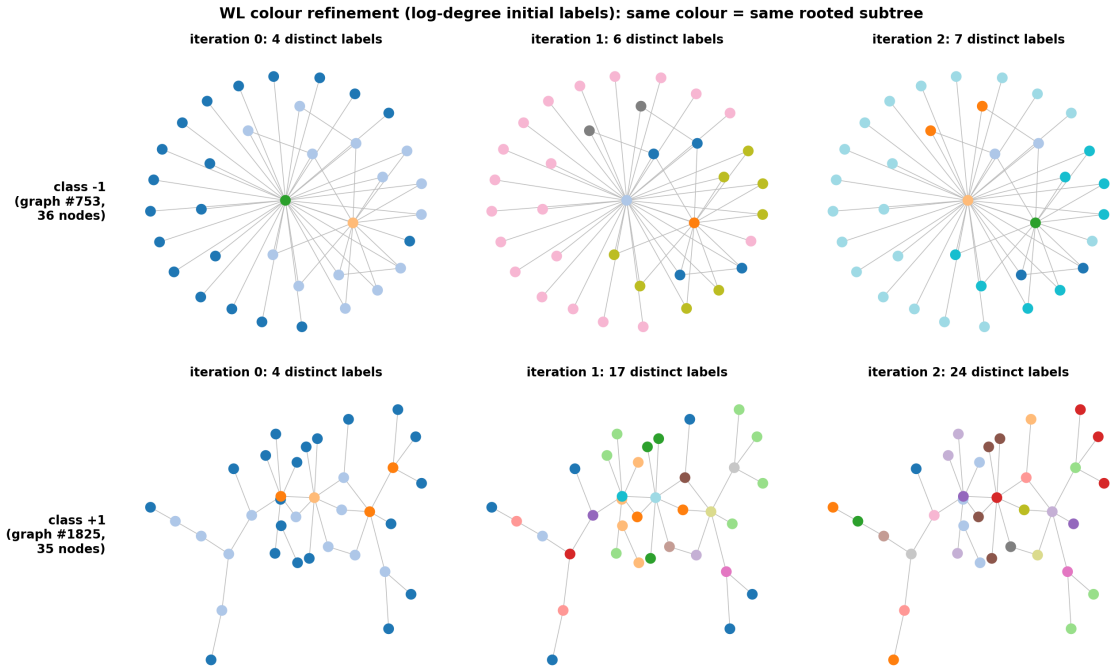


Figure 2. WL colour refinement (log-degree labels) on one thread per class at iterations 0, 1 and 2. The same colour means the same rooted subtree.

Figure 2 shows the refinement on one small thread from each class. The class -1 thread is star-shaped: a couple of hub users reply to almost everyone, so WL needs only 6–7 labels to describe it and most nodes share the label "leaf attached to a big hub". The class $+1$ thread is a branching conversation with chains of replies, and it already splits into 17 different rooted subtrees at iteration 1. The WL histogram $\phi(G)$ counts exactly these differences in how many nodes have which local shape.

5.3 Limitations of the raw-degree baseline on Reddit

With raw degree as the starting label, a leaf attached to a hub of degree 336 receives the label $(1, \{336\})$, which probably exists in no other thread. After one iteration only 40.2% of nodes carry a label that appears in at least 5% of the graphs. Labels that appear in only one graph add nothing to the similarity between *different* graphs, so the kernel mostly measures noise.

The dot product has a second weakness: it multiplies counts, so two large threads look similar simply because both contain many leaves, and big graphs dominate the comparison. These two weaknesses motivate our two improvements.

6. Improvement 1: Log-Binned Degree Labels

6.1 Idea

We change one at a time and keep WL itself untouched. The first change replaces the raw degree with a *logarithmic bin* of the degree as the starting label:

$$l^{(0)}(v) = 1 + \lfloor \log_2 d_v \rfloor \quad (d_v \geq 1), \qquad l^{(0)}(v) = 0 \quad (d_v = 0), \tag{4}$$

which places degrees into powers-of-two ranges.

Degree d_v	**Starting label**	**Meaning**
1	1	Leaf: a user who replied once
2–3	2	Lightly active user
4–7	3	Active user
8–15	4	Very active user
16–31, 32–63, ...	5, 6, ...	Hub users (13 distinct labels over the whole dataset)

Table 3. Logarithmic degree bins used as the initial WL label.

6.2 Why it helps

A user with 47 replies and a user with 48 replies no longer receive completely different labels. The bins keep the meaningful distinction (leaf, active user, hub) and discard the noise. The same leaf attached to a large hub now gets the label $(1, \{\text{bin } 9\})$, which is shared by every thread that contains a big hub.

To quantify this we measure how *shareable* the labels are. A WL label only adds to the similarity of two different graphs if both contain it, so for every node we count how many graphs contain that node's label. A node is *shared* if its label occurs in at least 2 graphs and *common* if it occurs in at least 5% of all graphs (≥ 100 graphs).

Quantity (after one WL iteration)	**Raw degree**	**Log-degree**
Distinct initial labels	566	13
Distinct labels at $h = 1$	71,893	15,393
Shared nodes (label in ≥ 2 graphs)	65.2%	98.8%
Common nodes (label in $\geq 5\%$ of graphs)	40.2%	75.6%
Accuracy, linear kernel, h tuned (%)	81.05	**88.30**

Table 4. Effect of binning the degree on label sharing and accuracy (10-fold nested CV, one repetition).

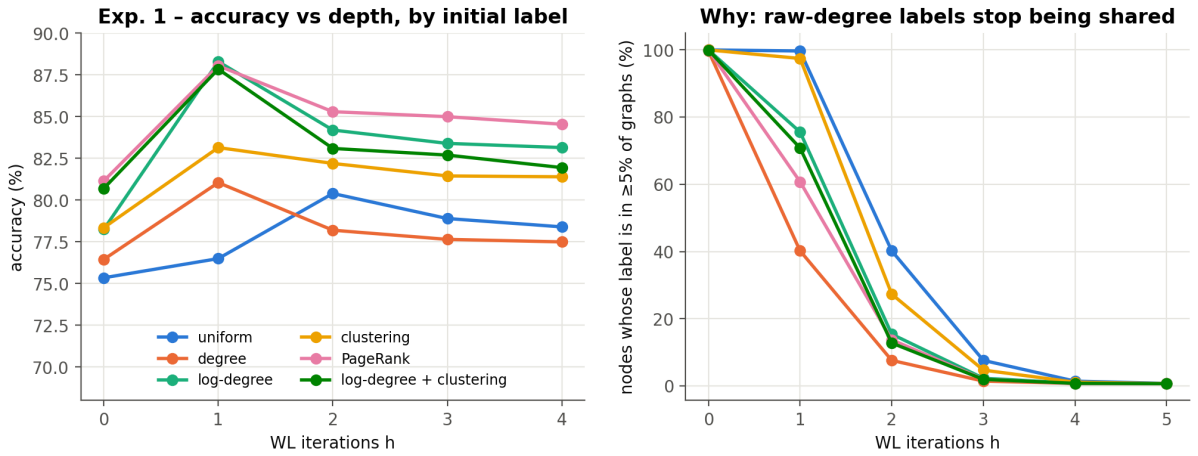


Figure 3. Accuracy versus WL depth for raw-degree and log-degree labels (left), and the share of nodes whose label occurs in at least 5% of graphs (right).

After one iteration, 75.6% of nodes carry a shareable label, compared with 40.2% for raw degree, while the number of distinct labels falls from 71,893 to 15,393. Binning adds about 7 accuracy points, the largest gain obtained from changing a single line of WL. The key point is that WL needs labels that *repeat across graphs*.

7. Improvement 2: Comparing WL Histograms - RBF Kernel

The second improvement changes Step 3 of WL, the way two label-count histograms are compared. Since $\phi(G)$ is a concatenation of per-iteration blocks, every kernel below is a sum over iterations, $K^{(h)} = \sum_{i=0}^{h} K_i$, so each iteration's Gram matrix is computed once and added up. We keep log-degree labels and Eq. 7.60 and change only the kernel.

7.1 RBF kernel on WL features

Instead of the dot product, we measure the distance between two WL histograms and convert it into a similarity. Because the squared distance can be written using the linear kernel alone, no new feature computation is needed:

$$\|\phi(G) - \phi(G')\|^2 = k(G,G) + k(G',G') - 2k(G,G'), \tag{5}$$

$$k_{\mathrm{RBF}}(G,G') = \exp\big(-\gamma\|\phi(G) - \phi(G')\|^2\big). \tag{6}$$

The bandwidth is set relative to the median squared distance, $\gamma = s/\operatorname{median}\|\phi - \phi'\|^2$ with scale $s \in \{0.25, 1, 4\}$, and is tuned in the inner cross-validation loop. Two graphs are now similar only if their histograms are *close*, not merely if both are large. The sum over iterations $K^{(h)} = \sum_i K_i$ is formed first, so the kernel remains a function of WL features only and the method stays pure WL.

7.2 Comparison of the kernels

Kernel on WL histograms	Accuracy (%)	± std	F1
Linear (original)	88.30	1.89	0.89
RBF (chosen)	**90.90**	2.17	0.91

Table 5. Kernel comparison with log-degree labels; h, C and γ are tuned in the inner loop (10-fold nested CV, one repetition).

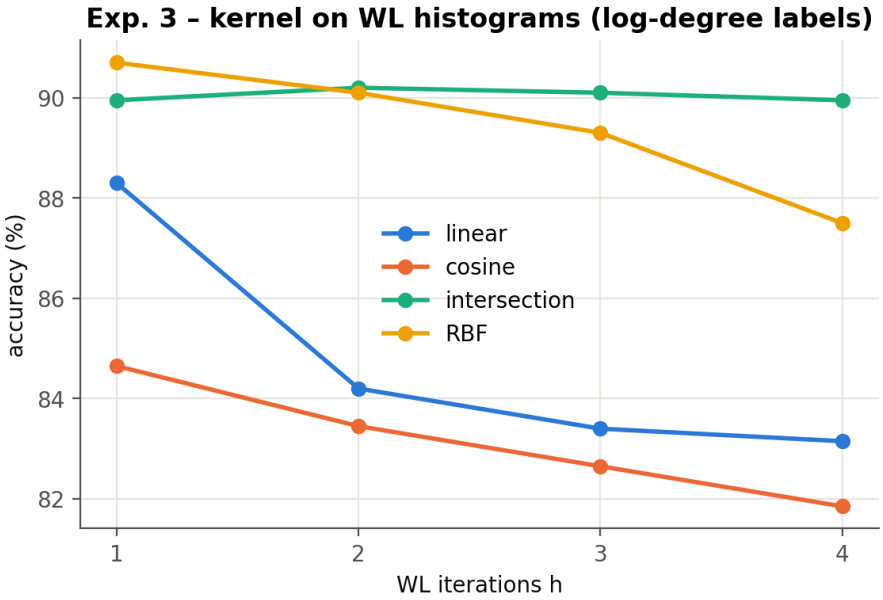


Figure 4. Accuracy versus WL depth for the linear, cosine-normalised and RBF kernels (log-degree labels).

RBF helps (+2.60 points versus linear). The linear kernel multiplies counts, so two huge threads look very similar simply because both have many leaves. The RBF kernel measures the distance $\|\phi - \phi'\|$, so graphs are similar only when their WL histograms are close, not merely when both are big.

7.3 Why the two changes fit together

Binning makes labels repeat across graphs, which gives the histograms meaningful overlap. The RBF kernel then compares those histograms in a way that is not dominated by graph size. Each change helps alone, but they address different weaknesses, so their gains largely add up (Section 9).

8. Implementation and Evaluation Protocol

8.1 Implementation and correctness testing

Everything mathematical (WL hashing, histograms, Gram matrices, RBF distances, metrics and the significance test) is written directly in NumPy. Only the SVM solver (scikit-learn `SVC` with a precomputed kernel) and plotting come from libraries. The implementation was verified before any experiment.

Test	**Result**
Hamilton Fig. 7.2	Iterations 1 and 2 reproduce the textbook partitions exactly.
Hamilton Fig. 7.3 (known failure)	A 6-cycle and two disjoint triangles obtain identical feature vectors, as 1-WL theory requires.
Permutation invariance	A 305-node thread and a random relabelling of it give identical WL features ($h = 5$).
Independent cross-check	Same WL partitions as NetworkX on 50 random REDDIT graphs, iterations 1–3.
Kernel validity	Linear, cosine and RBF Gram matrices are all positive semi-definite.

Table 6. Correctness tests, all passed.

While cross-checking we found that NetworkX's default WL hash joins label strings without a separator, so "1"+"12" and "11"+"2" collide. This makes its first hash non-injective when degrees have different digit counts. Our dictionary-of-tuples hash cannot collide, and for a fair comparison we gave NetworkX fixed-width labels. This illustrates Hamilton's point (§7.3.4) that WL depends on an injective hash.

8.2 Nested cross-validation

To avoid tuning on test data we use nested cross-validation. The outer loop is stratified 10-fold, repeated three times with different shuffles (30 outer folds) for the final results. On each outer training set, an inner stratified 5-fold loop selects the SVM cost C, the WL depth $h \in \{1,2,3,4\}$ and, for RBF, the bandwidth scale. The model is refitted with the chosen setting and evaluated on the untouched outer test fold. Accuracy and F1 (class $+1$ as positive) are computed by hand and reported as mean $\pm$ standard deviation over folds.

8.3 Significance test

Because outer folds share training data, a plain paired t-test is over-confident. Significance is therefore assessed with the corrected resampled t-test of Nadeau and Bengio (2003):

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{k} + \frac{n_{\text{test}}}{n_{\text{train}}}\right)\hat{\sigma}_d^2}}, \qquad d_j = \text{acc}_j^{\text{new}} - \text{acc}_j^{\text{baseline}}, \quad k = 30. \tag{7}$$

9. Final Results: Baseline to Best, One Change at a Time

This is the main experiment. Every row tunes its own hyper-parameters in the inner loop, and only one thing changes between the baseline and each row.

Setting	**Acc. (%)**	**± std**	**F1**	**Δ**	***p*-value**
Graph size only (RBF on $\log\|V\|$)	77.33	3.10	0.778	−4.42	2.2×10^{-4}
Baseline: degree + linear	81.75	2.46	0.826	—	—
+ log-degree labels (Improvement 1 only)	88.10	1.71	0.889	+6.35	1.1×10^{-8}
Degree + RBF (Improvement 2 only)	85.08	2.31	0.855	+3.33	6.5×10^{-4}
Ours: log-degree + RBF	**90.72**	1.83	0.910	+8.97	3.3×10^{-11}

Table 7. Final ablation, 10-fold × 3 nested CV (30 outer folds). p-values use the corrected resampled t-test against the baseline.

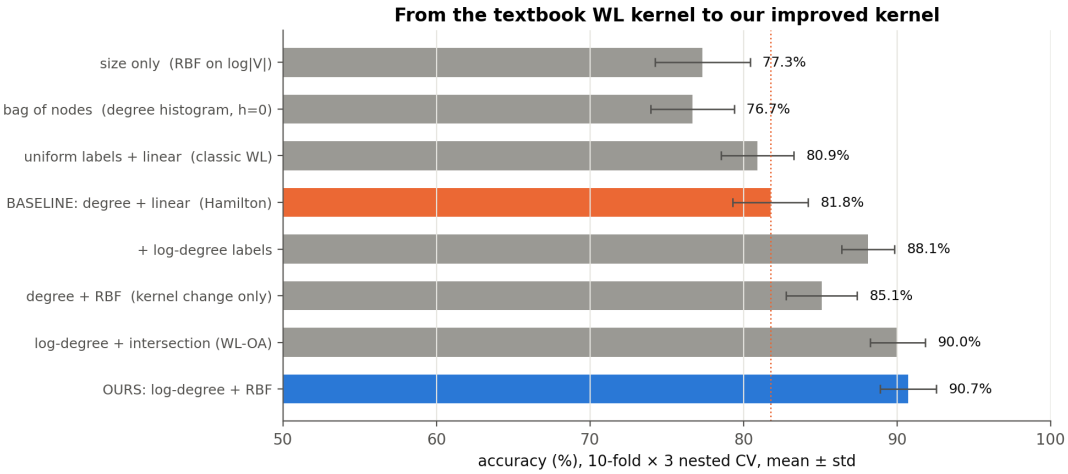


Figure 5. Accuracy of each setting (mean ± std). Orange: textbook baseline. Blue: our method.

9.1 What the results show

- **The two changes are complementary.** Labels alone add +6.35 points and the kernel alone adds +3.33, but together they add +8.97, cutting the error rate from 18.3% to 9.3% (a 49.1% relative reduction).

- **Variation shrinks.** The fold-to-fold standard deviation falls from 2.46 to 1.83.
- **The gain comes from structure, not size.** A classifier that sees only graph size scores 77.3%; the full model is about 13 points higher.
- **All improvements are significant** under the conservative corrected test.
- **Chosen settings.** The most frequently selected configuration for our method was $h = 1$, γ-scale 1.0, $C = 100$.

9.2 Confusion matrices

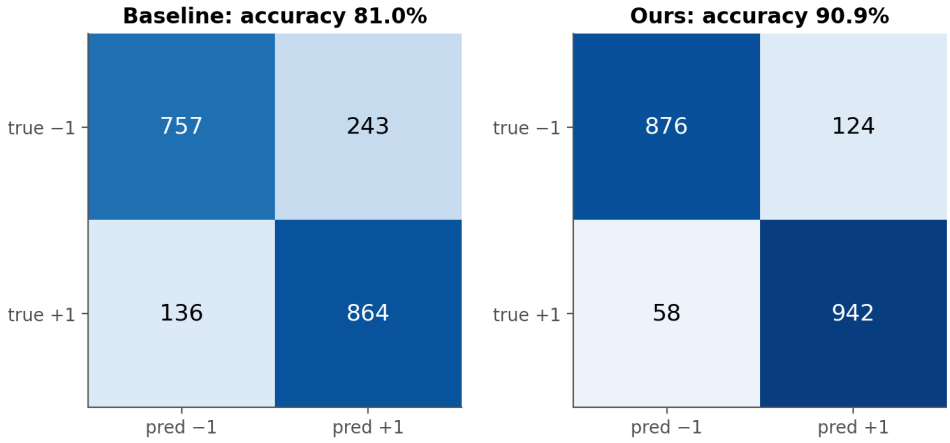


Figure 6. Confusion matrices for one CV repetition (each graph is predicted once while held out).

Total errors fall from 379 to 182. Both kinds of mistake are cut roughly in half (243 $\rightarrow$ 124 and 136 $\rightarrow$ 58).

9.3 What the kernels see

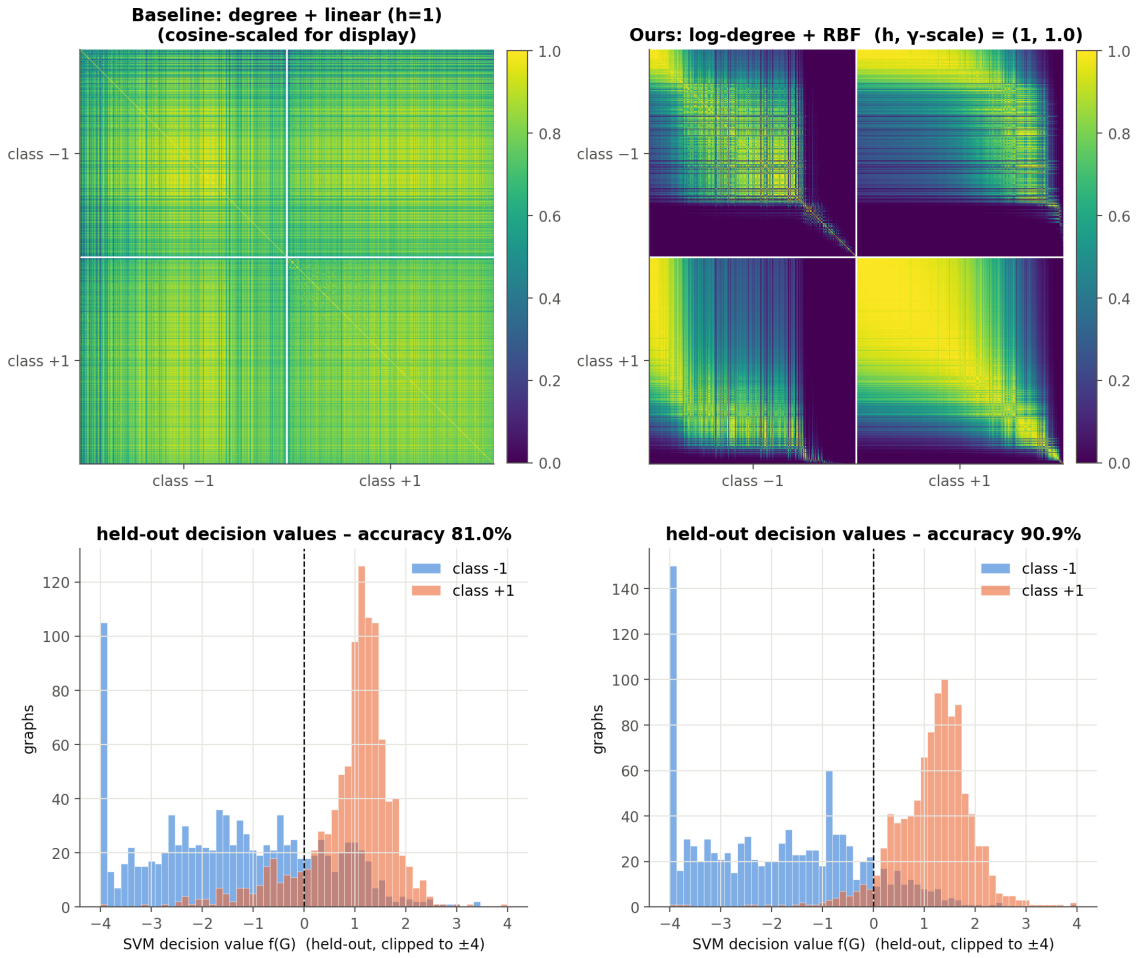


Figure 7. Gram matrices (top; graphs sorted by class, then size) and held-out SVM decision values (bottom) for the baseline and our kernel.

- **Baseline Gram matrix:** almost uniformly bright, so every thread looks fairly similar to every other and the SVM has little class structure to use.
- **Our Gram matrix:** clear block structure. Class $+1$ threads are highly similar to each other, large class -1 threads are similar mainly to one another, and only the smallest class -1 threads still resemble class $+1$ threads. Small threads carry less structure and remain the hardest to classify.
- **Decision values:** with our kernel the two classes are pushed apart on either side of $f(G) = 0$ with much less overlap.

10. Conclusion

We implemented the WL subtree kernel from first principles, validated it using established WL examples, a known failure case, permutation invariance and an independent implementation, and then improved it on REDDIT-BINARY with two small changes that leave the WL algorithm

untouched. Replacing raw degree with log-binned degree makes WL labels repeat across graphs, and replacing the dot product with an RBF kernel compares histograms by distance rather than by size. Accuracy rises from 81.75% to 90.72%, a 49% reduction in errors, and the result is statistically significant under a conservative test.

References

[1] W. L. Hamilton, *Graph Representation Learning*. Morgan & Claypool, 2020, §2.1.2 (Eq. 2.5, 2.6) and §7.3 (Eq. 7.60, Figs. 7.2, 7.3).

[2] N. Shervashidze, P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-Lehman graph kernels," *Journal of Machine Learning Research*, vol. 12, 2011.

[3] P. Yanardag and S. V. N. Vishwanathan, "Deep graph kernels," in *Proc. KDD*, 2015.

[4] C. Morris et al., "TUDataset: A collection of benchmark datasets for learning with graphs," 2020.

[5] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, 2003.